

Instituto Tecnológico de Mazatlán

Driver de software para un relevador

Práctica 4.1.1-A

Carrera:
Ingeniería en Sistemas Computacionales

Materia:
Sistemas Programables

Profesor:
Miguel Hernández Barrón

Alumnos:
Rosa Guadalupe Romero Navarro
Javier Ramón Quintero Valdez
Gabriel Alejandro Barrón Sarabia

Reporte de práctica: Driver de software para un relevador

1. Descripción del proyecto

Esta práctica (4.1.1-A) consiste en modelar por software un relevador mediante una clase en C++ llamada Relevador, que encapsula el estado del actuador, las órdenes que acepta y sus reglas de seguridad. Se utilizó un Arduino UNO R4 WiFi.

Un pin del Arduino entrega solo unos cuantos miliamperes, mientras que un motor pequeño puede pedir cientos al arrancar. Por eso el Arduino solo da la orden: la energía sale de otra fuente y un dispositivo intermedio conmuta la carga. Como no se contaba con un módulo de relevadores, se usaron dos actuadores que lo representan: un canal del puente H L298N con un motorreductor, que hace de "relevador electrónico" y se activa con nivel alto, y un LED conectado "al revés", que imita al optoacoplador de un módulo de relevador real y se activa con nivel bajo. La misma clase controla a los dos.

La clase recuerda si el actuador está encendido o apagado, traduce cada orden al nivel de pin correcto, impide conmutar más de una vez por segundo y garantiza que, al encender o reiniciar el sistema, la carga quede apagada. El programa principal nunca escribe directamente en los pines ni utiliza delay(): recibe órdenes por el Monitor Serie y se las pasa a los objetos.

2. Objetivos

- Abstraer el comportamiento de un relevador en un modelo de software con estados, órdenes y restricciones.
- Aplicar encapsulamiento en C++ para proteger el hardware de órdenes incorrectas.
- Resolver dentro del software la diferencia entre actuadores activos en alto y activos en bajo.
- Hacer cumplir un tiempo mínimo de 1000 ms entre cambios usando millis(), sin delay().
- Garantizar un estado seguro (apagado) al encender o reiniciar el sistema.
- Separar la parte de señal (Arduino) de la parte de potencia (fuente del motor).

3. Herramientas y material utilizado

Hardware:

- Arduino UNO R4 WiFi y cable USB-C
- Módulo puente H L298N
- Motorreductor DC tipo TT (3–6 V)
- Pila de 9 V para el motor (la práctica pide dos pilas 18650 en serie, ≈ 7.4 V; se sustituyeron porque el portapilas hacía falso contacto)
- LED y resistencia de 220 Ω

- Protoboard y cables jumper macho-macho y macho-hembra

Software:

- Arduino IDE
- Monitor Serie
- Tinkercad Circuits (simulación)

4. Diagrama

El circuito tiene una parte de potencia y una de señal. En la de potencia, el positivo de la pila se conecta a la terminal +12V del L298N y el negativo a su terminal GND, el motorreductor va a las salidas OUT1 y OUT2, y los jumpers ENA y 5V-EN del módulo se dejan puestos. La terminal +5V del L298N queda sin conectar. El GND del L298N se une al GND del Arduino para tener una tierra común.

En la parte de señal, el pin D7 del Arduino se conecta a IN1 del L298N e IN2 se conecta a GND, de modo que el motor solo enciende y apaga. El LED se conecta al revés: su ánodo va a 5V y su cátodo, a través de la resistencia de 220 Ω, al pin D6. Así el LED enciende cuando D6 está en nivel bajo.

Conexión	**Destino**
Positivo de la pila de 9 V	+12V del L298N
Negativo de la pila de 9 V	GND del L298N
GND del L298N	GND del Arduino (tierra común)
Motorreductor	OUT1 y OUT2
D7 del Arduino	IN1 del L298N (motor, activo en alto)
IN2 del L298N	GND
5V del Arduino	Ánodo del LED (pata larga)
Cátodo del LED	Resistencia de 220 Ω → D6 (LED, activo en bajo)

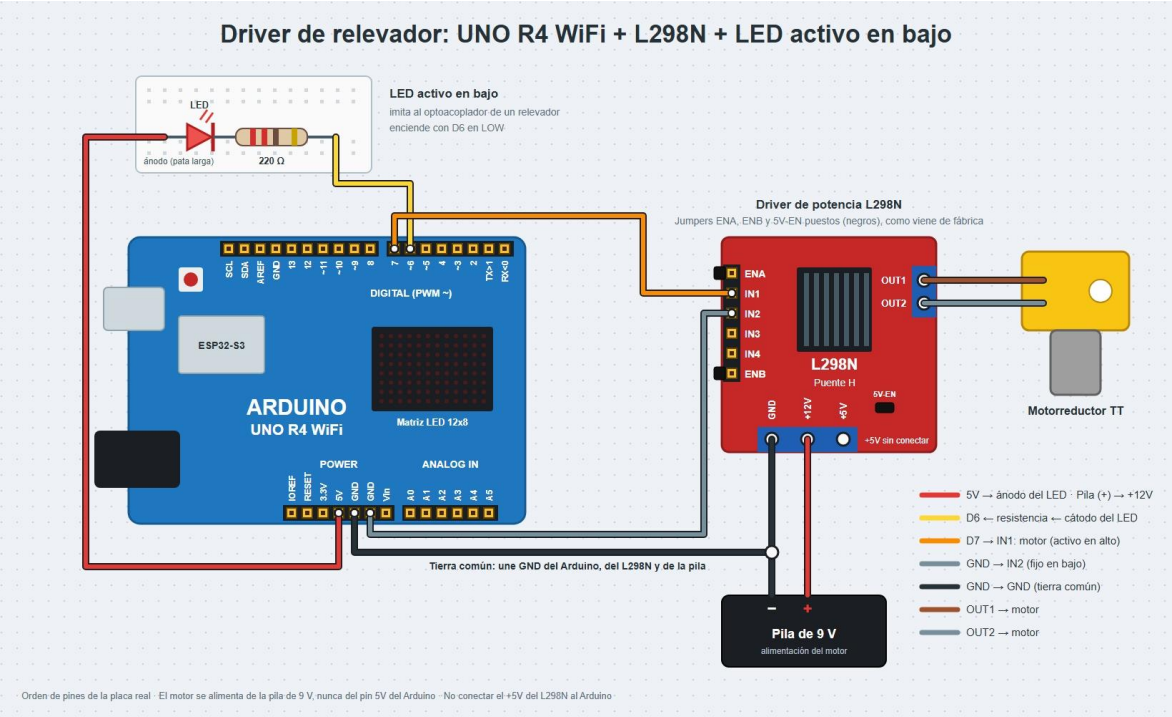


Diagrama de conexión del Arduino UNO R4 WiFi, el puente H L298N, el motorreductor y el LED activo en bajo.

Antes de programar se diseñó el diagrama de estados del relevador. Tiene dos estados: APAGADO, que es el estado inicial y seguro, y ENCENDIDO. Un cambio de estado solo se acepta si pasaron al menos 1000 ms desde el anterior; si la orden llega antes, se rechaza y el pin no cambia.

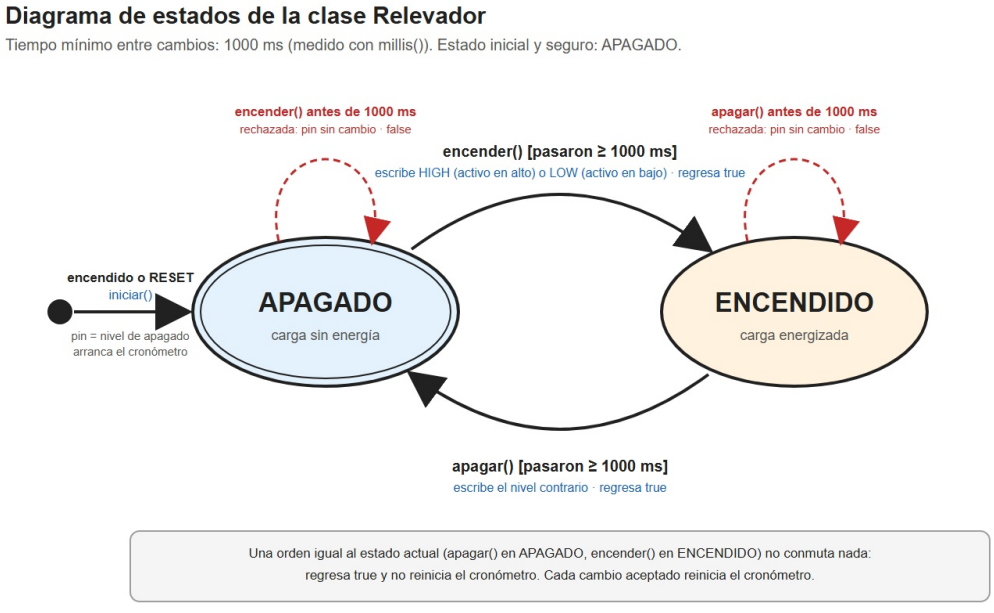


Diagrama de estados de la clase Relevador.

5. Código

El siguiente código define la clase Relevador, crea un objeto para el motor (D7, activo en alto) y otro para el LED (D6, activo en bajo), y atiende los comandos del Monitor Serie sin bloquear el programa:

```
/*
  Practica 4.1.1-A: Driver de software para un relevador.
  Arduino UNO R4 WiFi + modulo puente H L298N + motorreductor + LED.
  Equipo: Rosa Guadalupe Romero Navarro, Javier Ramon Quintero Valdez
          y Gabriel Alejandro Barron Sarabia.

  Motor (activo en ALTO): D7 -> IN1 del L298N; IN2 a GND; jumpers ENA y 5V-EN puestos.
  LED (activo en BAJO):   5V -> anodo del LED; catodo -> 220 ohm -> D6.
  Potencia: pila de 9 V a +12V y GND del L298N (la practica pide 2x18650, ~7.4 V).
  GND del L298N unido al GND del Arduino (tierra comun obligatoria).

  Comandos por el Monitor Serie (115200 baudios, cualquier ajuste de linea):
    M1 / M0  encender / apagar el motor
    L1 / L0  encender / apagar el LED
    E        estado de ambos
  Varias ordenes en una linea, separadas por coma o espacio, se ejecutan seguidas:
  "M1,M0" sirve para la prueba de seguridad 1 (la segunda debe rechazarse).

  El programa principal nunca escribe en los pines: todo pasa por la clase Relevador.
  No utiliza delay().
*/

#include <ctype.h>

const uint8_t PIN_MOTOR = 7;
const uint8_t PIN_FOCO = 6;              // LED externo (PIN_LED ya es el LED de la
placa, D13).
const uint32_t TIEMPO_MINIMO_MS = 1000;   // Entre dos cambios del mismo relevador.
const uint32_t FIN_DE_ORDEN_MS = 50;       // Silencio que cierra una orden sin fin de
linea.
const uint8_t LARGO_ORDEN = 8;



// ---------------------------------------------------------------- Modelo del
relevador
class Relevador {
  public:
    Relevador(uint8_t pin, bool activoEnBajo, uint32_t tiempoMinimo = TIEMPO_MINIMO_MS)
      : pin_(pin), activoEnBajo_(activoEnBajo), encendido_(false),
        ultimoCambio_(0), tiempoMinimo_(tiempoMinimo) {}

    // Estado seguro: el nivel de "apagado" se escribe antes y despues de volver
    // salida el pin, para que el actuador no parpadee al arrancar.
    void iniciar() {
      digitalWrite(pin_, nivel(false));
      pinMode(pin_, OUTPUT);
      digitalWrite(pin_, nivel(false));
      encendido_ = false;
      ultimoCambio_ = millis();             // El arranque cuenta como un cambio.
    }

    bool encender() { return cambiar(true); }
```

```
    bool apagar() { return cambiar(false); }
    bool estaEncendido() const { return encendido_; }

  private:
    uint8_t pin_;
    bool activoEnBajo_;
    bool encendido_;
    uint32_t ultimoCambio_;
    uint32_t tiempoMinimo_;

    // Traduce encendido/apagado al nivel del pin segun la logica del actuador.
    uint8_t nivel(bool encender) const {
      return (encender != activoEnBajo_) ? HIGH : LOW;
    }

    bool cambiar(bool encender) {
      if (encender == encendido_) {
        return true;                        // Ya esta asi: no hay nada que conmutar.
      }
      uint32_t ahora = millis();
      if (ahora - ultimoCambio_ < tiempoMinimo_) {
        return false;                       // Rechazada: el pin no cambia.
      }
      digitalWrite(pin_, nivel(encender));
      encendido_ = encender;
      ultimoCambio_ = ahora;
      return true;
    }
};



Relevador motor(PIN_MOTOR, false);          // L298N: activo en alto.
Relevador led(PIN_FOCO, true);              // LED al reves: activo en bajo.

char orden[LARGO_ORDEN + 1];
uint8_t largo = 0;
uint32_t ultimoCaracter = 0;



// ------------------------------------------------------------------ Monitor Serie
// Estas funciones no reciben un Relevador como parametro: Tinkercad declara las
// funciones al inicio del archivo, antes de la clase, y no reconoceria el tipo.
const char *textoEstado(bool encendido) {
  return encendido ? "ENCENDIDO" : "APAGADO";
}

void mostrarEstado() {
  Serial.print("Estado -> Motor: ");
  Serial.print(textoEstado(motor.estaEncendido()));
  Serial.print(" | LED: ");
  Serial.println(textoEstado(led.estaEncendido()));
}

void ordenar(bool esMotor, bool encender) {
  Relevador &r = esMotor ? motor : led;
  const char *nombre = esMotor ? "Motor" : "LED";
  bool yaEstaba = (r.estaEncendido() == encender);
  bool logrado = encender ? r.encender() : r.apagar();
  if (!logrado) {
    Serial.print("Orden rechazada (");
```

```
    Serial.print(nombre);
    Serial.println("): espere 1 s entre cambios");
    return;
  }
  Serial.print(nombre);
  Serial.print(yaEstaba ? " ya estaba " : " ");
  Serial.println(textoEstado(r.estaEncendido()));
}

void ejecutar(const char *texto) {
  if (strcmp(texto, "M1") == 0)      ordenar(true, true);
  else if (strcmp(texto, "M0") == 0) ordenar(true, false);
  else if (strcmp(texto, "L1") == 0) ordenar(false, true);
  else if (strcmp(texto, "L0") == 0) ordenar(false, false);
  else if (strcmp(texto, "E") == 0)  mostrarEstado();
  else {
    Serial.print("Comando desconocido: ");
    Serial.print(texto);
    Serial.println("  (use M1, M0, L1, L0 o E)");
  }
}

void cerrarOrden() {
  if (largo > 0) {
    orden[largo] = '\0';
    ejecutar(orden);
  }
  largo = 0;
}

// Junta los caracteres sin bloquear. La orden termina con fin de linea, coma o
// espacio o, si el Monitor Serie esta en "Sin ajuste de linea", tras 50 ms sin
caracteres.
void leerSerie() {
  while (Serial.available() > 0) {
    char c = (char)Serial.read();
    ultimoCaracter = millis();
    if (c == '\n' || c == '\r' || c == ',' || c == ' ') {
      cerrarOrden();
    } else if (largo < LARGO_ORDEN) {
      orden[largo++] = (char)toupper((unsigned char)c);
    }
  }
  if (largo > 0 && millis() - ultimoCaracter >= FIN_DE_ORDEN_MS) {
    cerrarOrden();
  }
}



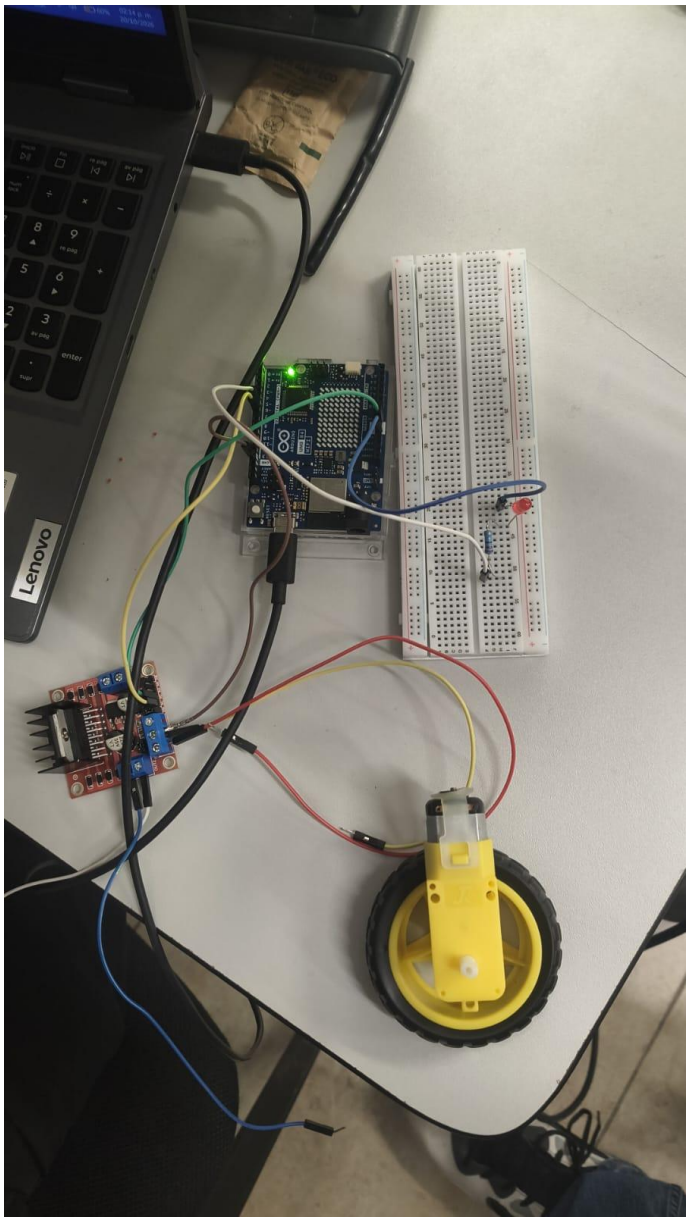
// ------------------------------------------------------------------ Programa principal
void setup() {
  motor.iniciar();                          // Primero el estado seguro, luego lo
demas.
  led.iniciar();
  Serial.begin(115200);
  uint32_t inicio = millis();
  while (!Serial && millis() - inicio < 2000) {
    // Espera al Monitor Serie hasta 2 s, sin delay(); los actuadores ya estan
apagados.
  }
  Serial.println();
  Serial.println("Practica 4.1.1-A: driver de software para un relevador");
```

```
  Serial.println("Sistema iniciado en estado seguro.");
  mostrarEstado();
  Serial.println("Comandos: M1, M0 (motor) | L1, L0 (LED) | E (estado) | varios:
M1,M0");
}

void loop() {
  leerSerie();
}
```

6. Montaje físico

Como parte de la práctica se realizó el armado físico del circuito siguiendo el diagrama: el Arduino UNO R4 WiFi conectado por USB a la computadora, el módulo L298N con la señal de D7 en IN1 y el motorreductor con su llanta en las salidas del puente H, y el LED con su resistencia en la protoboard. En la foto ya se había retirado la pila de 9 V que alimentaba el motor.



Montaje físico del circuito con el Arduino, el puente H L298N, el motorreductor y el LED.

7. Metodología

Primero se armó la parte de potencia con el USB desconectado: la pila a las terminales +12V y GND del L298N, el motorreductor a OUT1 y OUT2, y la tierra común entre el L298N y el Arduino. Después se conectó la parte de señal: D7 a IN1, IN2 a GND y el LED con su resistencia entre 5V y D6. Antes de energizar se revisó la polaridad de la pila.

Antes de programar se dibujó el diagrama de estados del relevador, con el estado inicial APAGADO y la regla del tiempo mínimo. Con base en él se programó la clase Relevador con sus datos privados (pin, lógica activa en bajo, estado, momento del último cambio y tiempo mínimo) y sus métodos públicos iniciar(), encender(), apagar() y estaEncendido().

Después se crearon los dos objetos de la misma clase y se programó la lectura del Monitor Serie para aceptar los comandos M1, M0, L1, L0 y E. Finalmente se realizaron las pruebas de seguridad: enviar M1 y M0 seguidos para comprobar el rechazo de órdenes demasiado rápidas, y presionar RESET con el motor encendido para comprobar que todo arranca apagado. Las pruebas del Monitor Serie que se presentan se hicieron en la simulación de Tinkercad, con el mismo código en un Arduino UNO R3.

8. Explicación del funcionamiento

La clase Relevador. Guarda como datos privados el pin que controla, si el actuador es activo en bajo, su estado actual, el momento del último cambio medido con millis() y el tiempo mínimo entre cambios, de 1000 ms. El resto del programa no puede modificar esos datos; solo puede usar los métodos públicos.

Activo en alto y activo en bajo. El método privado nivel() traduce la orden al nivel del pin: para encender un actuador activo en alto escribe HIGH, y para encender uno activo en bajo escribe LOW. Para apagar escribe el nivel contrario. Esta traducción existe en un solo lugar del programa, así que el motor y el LED se controlan con la misma clase cambiando solo un parámetro al crear cada objeto.

El tiempo mínimo entre cambios. Antes de conmutar, el método cambiar() calcula cuánto tiempo pasó desde el último cambio. Si es menor que 1000 ms, regresa false y termina sin llegar a digitalWrite(), por lo que el pin no cambia. Si la orden coincide con el estado actual (por ejemplo, apagar un actuador que ya está apagado), no hay nada que conmutar: regresa true y no reinicia el cronómetro.

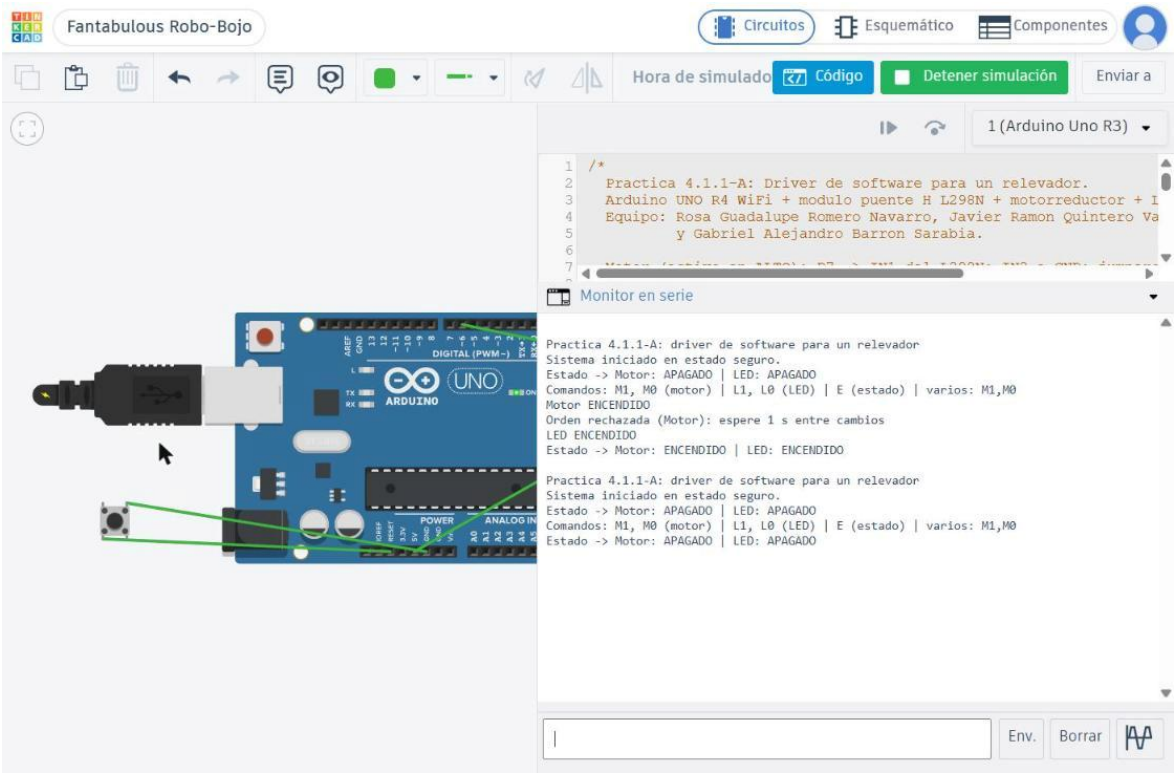
El estado seguro. El método iniciar() escribe el nivel de apagado, configura el pin como salida y vuelve a escribir el nivel de apagado, para que el actuador no parpadee al arrancar. En el setup() lo primero que se hace es iniciar los dos actuadores, antes incluso de abrir el puerto serie. El arranque cuenta como un cambio, así que durante el primer segundo después de un RESET el sistema rechaza las órdenes.

El Monitor Serie. El loop() solo llama a leerSerie(), que junta los caracteres sin bloquear el programa y ejecuta la orden al recibir un fin de línea, una coma o un espacio. Esto permite enviar varias órdenes en una sola línea, como "M1,M0", para la prueba de seguridad. Cada orden responde con un mensaje: el nuevo estado del actuador o "Orden rechazada: espere 1 s entre cambios".

Compatibilidad con Tinkercad. Tinkercad declara automáticamente las funciones al inicio del archivo, antes de la clase. Por eso las funciones del Monitor Serie no reciben un objeto Relevador como parámetro y el programa no incluye Arduino.h de forma explícita. El mismo código compila en el Arduino IDE para el UNO R4 WiFi.

9. Monitor Serie

La siguiente captura se tomó en la simulación de Tinkercad, con el mismo código en un Arduino UNO R3. El circuito simulado incluye el LED activo en bajo en D6 y un pulsador entre RESET y GND que hace de botón RESET. Tinkercad no tiene el módulo L298N, así que el estado del motor se comprueba en el Monitor Serie.



Monitor Serie de la simulación con el arranque seguro, el rechazo de órdenes demasiado seguidas y el reinicio con RESET.

10. Análisis de resultados

Al arrancar, el programa reportó "Sistema iniciado en estado seguro" con el motor y el LED apagados, lo que confirma que iniciar() deja ambos actuadores en su estado seguro antes de aceptar órdenes.

Prueba de seguridad 1. Al enviar M1,M0 en una sola línea, el motor se encendió y la segunda orden se rechazó con el mensaje "Orden rechazada (Motor): espere 1 s entre cambios". Las dos órdenes llegaron con una diferencia de milisegundos, mucho menos que el tiempo mínimo de 1000 ms, así que la clase protegió al actuador y el motor siguió encendido. Después, L1 encendió el LED y el comando E reportó los dos actuadores ENCENDIDOS.

Prueba de seguridad 2. Con el motor y el LED encendidos se presionó RESET. El programa se reinició y volvió a mostrar el mensaje de arranque en estado seguro, y el comando E confirmó los

dos actuadores APAGADOS. Al reiniciar, el microcontrolador pierde el estado anterior y el setup() vuelve a llamar a iniciar(), así que la carga nunca queda energizada después de un reinicio.

Prueba de inversión. Si el objeto del LED se crea por error como activo en alto, el modelo queda al revés de la realidad. Al arrancar, iniciar() escribe LOW creyendo que apaga el LED, pero como el ánodo está en 5V, ese LOW lo enciende. Después, L1 escribe HIGH y el LED se apaga mientras el programa reporta "LED ENCENDIDO". Con un relevador real, la carga quedaría energizada justo después de cada reinicio, que es lo contrario al estado seguro. Esto muestra que el parámetro activo en alto o en bajo debe coincidir con el hardware.

11. Preguntas de reflexión

1. ¿Por qué el LED se enciende con LOW y no con HIGH? Explica el recorrido de la corriente.

Porque el ánodo del LED está conectado de forma fija a los 5V del Arduino. La corriente recorre 5V → ánodo → cátodo → resistencia de 220 Ω → pin D6, y para que circule el otro extremo tiene que estar a un voltaje menor. Cuando D6 está en LOW (0 V) hay 5 V de diferencia, el pin absorbe la corriente y el LED se enciende. Cuando D6 está en HIGH (5 V), los dos extremos están al mismo voltaje, no circula corriente y el LED se apaga. Lo mismo pasa con el optoacoplador de un módulo de relevadores, cuyo LED interno está conectado igual.

2. ¿Qué ventaja tiene que la regla del tiempo mínimo viva dentro de la clase y no en el loop()?

Que la regla no se puede olvidar ni saltar. Cualquier parte del programa que mande encender o apagar pasa por la clase y queda protegida, sin repetir la comprobación en cada lugar. Además, cada objeto lleva su propio cronómetro, así que el motor y el LED respetan su tiempo de forma independiente, y el loop() se queda corto y fácil de leer. Si la regla cambia, se modifica en un solo lugar.

3. ¿Por qué un relevador real no sirve para controlar velocidad con PWM?

Porque es un interruptor mecánico: una bobina mueve un contacto que tarda varios milisegundos en cerrar o abrir y rebota al hacerlo. El PWM del Arduino conmuta cientos de veces por segundo, una velocidad que el relevador no puede seguir. Además, cada conmutación desgasta los contactos y genera chispas, y un relevador típico dura del orden de cien mil operaciones con carga, así que a esa frecuencia se acabaría en minutos. Para regular velocidad se usan interruptores electrónicos, como los transistores del puente H o un MOSFET.

4. ¿Por qué el estado seguro es "apagado"? Da un ejemplo industrial donde convenga lo contrario (contacto NC).

Porque si el sistema se reinicia, pierde energía o falla el control, lo menos peligroso suele ser que la máquina se detenga: un motor que arranca solo después de un RESET puede lastimar a quien

trabaja cerca. Lo contrario conviene cuando detener la carga es lo peligroso. Por ejemplo, el extractor de gases de una mina o de un laboratorio químico, o la bomba de enfriamiento de un horno industrial, deben seguir funcionando aunque falle el control. Por eso se conectan al contacto normalmente cerrado (NC), que queda cerrado cuando el relevador pierde la energía.

5. ¿Por qué es obligatoria la tierra común entre el portapilas y el Arduino?

Porque un voltaje siempre se mide respecto a una referencia. Cuando D7 pone 5 V en IN1, esos 5 V son respecto a la tierra del Arduino, pero el L298N interpreta la señal respecto a su propia tierra, que es el negativo de la pila. Si las tierras no están unidas, la señal no tiene camino de regreso y el L298N no tiene una referencia para saber si IN1 está en alto o en bajo: la entrada queda flotando y el motor puede no responder o encender al azar.

6. Si mañana cambiaras el L298N por un módulo de relevador real, ¿qué partes del programa tendrías que modificar?

Solo la línea donde se crea el objeto: el pin y si es activo en bajo. La mayoría de los módulos de relevadores con optoacoplador son activos en bajo, así que quedaría como Relevador motor(PIN_MOTOR, true);. La clase, sus reglas y el loop() no cambian, porque el resto del programa le habla al relevador con órdenes (encender, apagar) y no con niveles de pin. Lo que sí cambiaría es el cableado: el módulo se alimenta con VCC y GND, recibe la señal en IN y la carga se conecta en COM y NO.

12. Conclusiones

Gabriel

Esta práctica me ayudó a entender que un actuador no se controla con pines sueltos repartidos por el programa, sino con un modelo de software que le da significado a las órdenes. Al encapsular el relevador en una clase, el resto del programa solo dice encender o apagar y no necesita saber qué nivel de pin requiere cada dispositivo. También me quedó claro por qué la regla del tiempo mínimo debe vivir dentro de la clase: así ninguna parte del programa puede saltársela.

Javier

Lo que más me llamó la atención fue que la diferencia entre activo en alto y activo en bajo se resuelve por completo en el software. El motor y el LED funcionaron con la misma clase cambiando un solo parámetro, y la prueba de inversión mostró lo que pasa si ese parámetro no coincide con el hardware: el programa cree que el actuador está apagado cuando en realidad está encendido. También reforcé la importancia de la tierra común entre la fuente del motor y el Arduino.

Rosa

Esta práctica me permitió comprender que el software también debe proteger al hardware. El tiempo mínimo entre cambios cuida las partes mecánicas de un relevador real, y el estado seguro al arrancar evita que la carga se energice sola después de un reinicio, como comprobamos con la

prueba del RESET. Además entendí por qué la parte de señal y la de potencia deben estar separadas: el Arduino solo da la orden y la energía del motor sale de otra fuente.